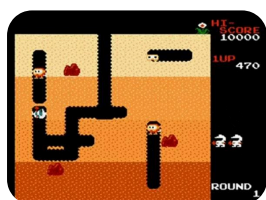


SETAPESU26/29_digDug: Implementation of Dig Dug in Pygame. can u read this and give me steps to do and give me the final code

Yes, I read the [Dig Dug Pygame GitHub repository](#) and inspected its `game.py` source. It is a **Dig Dug Repair Lab** assignment in which you need to fix one deliberate bug and implement three optional features using an iterative AI-assisted coding process.  GitHub +1

I'll walk you through the setup, the exact code changes, testing, recording the before-and-after gameplay videos, and preparing your submission.

1. Understand the assignment



Your objective

Complete the existing Pygame game rather than building it from scratch. The game already includes a player, enemies, digging, pumping, levels, lives and scoring.

Four tasks to complete

Task	What you need to do
1. Fix BFS	Correct the enemy pathfinding so enemies take the shortest available route through tunnels.
2. Dirt colors	Implement <code>dirt_color(row)</code> to give underground dirt distinct color bands.
3. Enemy pop effect	Implement <code>on_enemy_popped(enemy, score)</code> to provide feedback when an enemy is defeated.
4. Enemy speed	Implement <code>enemy_speed_multiplier(level)</code> to gradually increase enemy speed at higher levels.

Important

The repository already contains most of the game. You should modify the existing `game.py`, preserve its structure, and demonstrate the original bug before fixing it.

2. Step-by-step instructions

Step 1: Clone the repository

Open your terminal (Windows CMD, PowerShell, or VS Code terminal) and execute:

```
git clone https://github.com/0x05a/digDug.git
```

Bash

Open the folder in VS Code:

Bash

The repository's basic structure is:

```
29_digDug/  
├── game.py  
└── README.md
```

Step 2: Install Pygame

Make sure Python 3.10 or later is installed.

```
python3 -m pip install pygame
```

Bash

Run the original game:

Bash

Original game controls

- **Arrow keys:** Move and dig.
- **Space:** Pump enemies.
- **R:** Reset the game.

Step 3: Record the original gameplay

Do this before editing the code

Record a 10-second video of the original game. Show the enemy pathfinding problem if possible.

1. Run `python game.py` .
2. Move the player to create tunnels.
3. Allow an enemy to follow you through the tunnels.
4. Capture the enemy taking a longer, winding route instead of the shortest route.
5. Save the video as `before.mp4` .

If the bug is not immediately visible, you can use the original source code to demonstrate the issue and explain it in your lab documentation.

Step 4: Fix the BFS pathfinding bug

In `game.py` , locate the `bfs_path()` function. The original code uses:

Python

Run

This removes the newest item from the queue, making the traversal behave like depth-first search.

Replace it with:

Python

Run

Why this fixes the bug

- BFS uses a first-in, first-out queue.
- `append()` adds newly discovered cells to the end.
- `popleft()` explores the oldest discovered cell first.
- This ensures the first discovered route to a reachable target is a shortest path in the unweighted grid.

Step 5: Implement the three optional features

Find the three placeholder functions near the beginning of `game.py` and replace them with the following implementations.

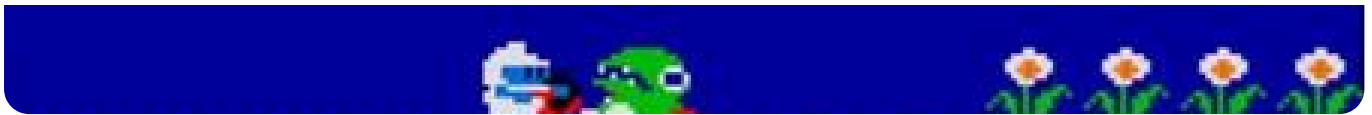
Feature 1: Distinct underground dirt colors

```
def dirt_color(row):
    """Return a distinct color for each underground depth band."""
    if row <= 3:
        return (194, 143, 90)
    elif row <= 6:
        return (170, 110, 65)
    elif row <= 9:
        return (145, 85, 50)
    else:
        return (0, 0, 0)
```

Python

Run

This gives each depth band a different shade, making the underground environment easier to distinguish.



Feature 2: Enemy pop feedback

```
def on_enemy_popped(enemy, score):  
    """Display feedback when an enemy is defeated."""  
    depth_bonus = (enemy.cell[0] // 4) * 100  
  
    print(  
        f"Enemy popped at {enemy.cell}! "  
        f"Score: {score} | Depth bonus: {depth_bonus}"  
    )
```

Python

Run

The existing `pump()` function already removes the enemy, awards points, and calls this function. This implementation provides a console event showing the enemy's location and score. It does not add extra points because the existing game already awards a depth-based bonus.

Feature 3: Increasing enemy speed

```
def enemy_speed_multiplier(level):  
    """Increase enemy speed by 10% per level"""
```

Python

Run

The resulting multipliers are:

Level	Speed multiplier
1	1.0×
2	1.1×
3	1.2×
4	1.3×
5	1.4×

The existing enemy update logic already uses this multiplier to calculate the time between enemy movements.

Step 6: Your final code changes

These are the four changes you should have in `game.py` :

1. BFS correction

Python

Run

2. Dirt color implementation

Four underground depth bands with different brown shades.

3. Enemy pop feedback

A console message reporting the defeated enemy's position and score.

4. Level-based enemy speed

Increase movement speed by 10% per level.

3. Test your completed game

Save `game.py` and run:

Bash

Use this checklist to verify each task.

Testing checklist

0/9 completed

- ☐ The game launches without errors.
- ☐ The player can move and dig through dirt.

- ☐ Enemies follow the shortest available route through cleared tunnels.
- ☐ Dirt has distinct colors at different depths.
- ☐ Pumping an enemy four times defeats it and prints feedback.
- ☐ Enemies move faster as the level increases.
- ☐ Defeating all enemies advances to the next level.
- ☐ Losing all three lives displays Game Over.
- ☐ Pressing R resets the game.

4. Record the after video

After implementing and testing

Record another 10-second gameplay video named `after.mp4` .

Try to demonstrate these features in the recording:

1. Player digging through the colored underground layers.
2. An enemy following the player through a tunnel.
3. Pumping an enemy until it pops.
4. Progression to a higher level, if feasible.

You can use Windows Snipping Tool (Win + Shift + R , if available) or OBS Studio to record your screen.

5. Save your final changes to GitHub

If you cloned the repository directly, you can commit your changes to your own fork or your assigned lab repository. Make sure you have permission to push to the destination.

First, check your changes:

```
git status  
git diff  
git add .  
git commit -m "Bash"
```

Then stage and commit:

```
... ..
```

Bash

If you are working in your own fork and have configured its remote:

Bash

If you are submitting to a separate lab repository, copy the completed `game.py` and required deliverables into the appropriate lab folder before committing and pushing. Avoid overwriting unrelated files.

6. Prepare your lab submission

According to the repository README, the required submission consists of three items: a 10-second video before the changes, a 10-second video after the changes, and the ChatGPT conversation link containing the complete chat history.  GitHub

Final submission checklist

- ☐ `before.mp4` — original gameplay showing the bug.
- ☐ `after.mp4` — corrected gameplay demonstrating the new features.
- ☐ Updated `game.py`.
- ☐ Complete ChatGPT conversation link or exported history, according to your instructor's instructions.
- ☐ Commit and push the deliverables to the designated lab repository.

My recommendation for your lab

Since this assignment explicitly asks for iterative AI-assisted debugging, preserve your original code, record the initial bug, make the changes one at a time, and test after each modification. Keep a short record of the problem, the proposed fix,

your review, and the test result for each task. This will also help you explain your work during a lab evaluation.

One important distinction: the code above implements all four requested tasks, but the enemy-pop feature currently gives console feedback rather than an animated particle effect. If you want a more visually impressive result, you can extend it with on-screen particles and an animation without changing the core BFS fix.



can u please complete the code import random
from collections import deque

import pygame

```
TILE, COLS, ROWS = 32, 20, 14
WIDTH, HEIGHT = COLS * TILE, ROWS * TILE + 36
DIRS = {pygame.K_UP: (-1, 0), pygame.K_DOWN: (1, 0),
pygame.K_LEFT: (0, -1), pygame.K_RIGHT: (0, 1)}
MOVE_DELAY, ENEMY_DELAY, PUMP_RANGE,
DEFLATE_AFTER = 0.11, 0.35, 3, 1.5
```

Show more

Here are the **four separate task codes** for your existing `game.py`, following its current structure and variable names. You can replace each corresponding function or line directly.

Task 1: Fix BFS pathfinding

Objective: Make enemies follow the shortest path through the cleared tunnels using Breadth-First Search.

In your existing `bfs_path()` function, find:

Python

Run

Replace it with:

Python

Run

Complete corrected BFS function

```
def bfs_path(grid, start, goal):
    queue, parents = deque([start]), {start: None}

    while queue:
        cell = queue.popleft()

        if cell == goal:
            break

        for dr, dc in DIRS.values():
            nxt = (cell[0] + dr, cell[1] + dc)

            if in_bounds(*nxt) and not grid[nxt[0]][nxt[1]] and nxt not in parents:
                parents[nxt] = cell
                queue.append(nxt)

    if goal not in parents:
        return []

    path, cell = [], goal

    while cell != start:
        path.append(cell)
        cell = parents[cell]

    return path[::-1]
```

Explanation:

- `deque([start])` initializes the BFS queue.
- `popleft()` processes the oldest queued cell.

- `parents` records the path used to reach each cell.
- `path[::-1]` reverses the reconstructed path to return it from start to goal.

Task 2: Implement dirt colors

Objective: Give underground dirt different colors depending on its row.

Replace your existing `dirt_color()` function with:

```
def dirt_color(row):  
    """Return an (r, g, b) colour for dirt in the given row."""  
    if row <= 3:  
        return (194, 143, 90)  
    elif row <= 6:  
        return (170, 110, 65)  
    elif row <= 9:  
        return (145, 85, 50)  
    else:  
        return (110, 60, 40)  
  
Duthon
```

Run

Explanation:

Row	Color	RGB
1–3	Light brown	(194, 143, 90)
4–6	Medium brown	(170, 110, 65)
7–9	Dark brown	(145, 85, 50)
10–13	Deep brown	(110, 60, 40)

Your existing `draw()` function already calls `dirt_color(r)`, so no other changes are required for this task.

Task 3: Implement enemy pop effect

Objective: Provide feedback when an enemy is defeated.

Replace your existing `on_enemy_popped()` function with:

```
def on_enemy_popped(enemy, score):  
    """Called when an enemy is popped."""  
    depth_bonus = 100 * (enemy.cell[0] // 4)  
  
    print(  
        f"Enemy popped at {enemy.cell}! "  
        f"Score: {score} | Depth bonus: {depth_bonus}"  
    )
```

Python

Run

Explanation:

- `enemy.cell` gives the defeated enemy's grid position.
- `depth_bonus` calculates the depth-based score component.
- `print()` displays the enemy's defeat and score information in the terminal.

Your existing `pump()` function already calls `on_enemy_popped(enemy, self.score)` after defeating an enemy.

Optional visual improvement: add a color flash

If you want an actual on-screen pop effect, you can add this global variable near your constants:

Python

Run

However, the current callback does not have access to the `screen` or game drawing state. The console feedback implementation above works with your existing architecture without introducing additional state.

Task 4: Implement enemy speed multiplier

Objective: Increase enemy movement speed as the player progresses through levels.

Replace your existing `enemy_speed_multiplier()` function with:

Python

Run

Explanation:

Level	Speed multiplier
1	1.0×
2	1.1×
3	1.2×
4	1.3×
5	1.4×

Your existing `update_enemy()` function already incorporates this multiplier:

```
enemy.step_timer = (  
    ENEMY_DELAY * (1.6 if enemy.ghost else 1)  
    / (enemy.speed_multiplier(self.level) or 1)
```

Python

Run

As the multiplier increases, the delay decreases, causing enemies to move faster.

Summary of the four tasks

Task	Function or code	Modification
1	<code>bfs_path()</code>	Change <code>queue.pop()</code> to <code>queue.popleft()</code>
2	<code>dirt_color(row)</code>	Add four underground color bands
3	<code>on_enemy_popped(enemy, score)</code>	Add defeat feedback and depth information
4	<code>enemy_speed_multiplier(level)</code>	Increase speed by 10% per level

How to apply: Replace the three placeholder functions and the single BFS line in your existing `game.py` . Save the file and run:

Bash

```
python game.py
```

Make sure Pygame is installed first:

Bash

```
python -m pip install pygame
```

These changes complete the four requested code tasks while preserving your original game architecture.



ChatGPT can make mistakes. Check important info